

Programmable In-Network Timing Obfuscation for DNP3

Anonymous Submission

Abstract—An adversary watching a power-grid control network can tell which device it is looking at from how long that device takes to answer, because vendors implement the same function differently. The relay cannot be modified, and existing traffic obfuscation targets a different feature: it reshapes flow-level sizes and inter-packet times, whereas what identifies the device here is one interval inside a single transaction.

We place the defense in the network instead. A programmable switch in front of the outstation holds the device's acknowledgment and its response, releasing each at a deadline computed from one common instant, the moment the outstation acknowledges the request. Because both deadlines come from that one instant, the device's own processing interval cancels from the gap between the two releases whenever the response arrives in time, so the interval an observer measures is one the operator sets. The switch forwards the packets it received, so both endpoints run unmodified and no DNP3 byte changes.

We implement this on an Intel Tofino-1 and evaluate it against one physical SEL-751A protective relay over 63,360 exchanges. The measured interval settles on the configured value: its median is the configured 4 ms and its sample variance falls from 6.808 to 0.394 ms². A classifier trained on that interval before deployment is left at chance across READ, SELECT and OPERATE. An adversary who retrains recovers much of that from a second interval the mechanism does not target but does make more informative, and that residual bounds what the design achieves. The evaluation covers transaction-class timing for a single device at a single configuration, and does not address discrimination between device models.

Index Terms—traffic obfuscation, device fingerprinting, DNP3, programmable switches.

I. INTRODUCTION

Device fingerprinting has been an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and to design effective, stealthy attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to inflict physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities,

it is widely believed that adversaries stay in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies present network traffic obfuscation [3], [4]. Device fingerprinting targeting general computing environments generally relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5]. Consequently, existing traffic obfuscation often focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [6]. Since manipulating communication networks can introduce runtime overhead, recent works have begun to offload traffic obfuscation onto programmable network switches, which change communication patterns at much higher line rates than CPUs [7]–[9].

Unfortunately, these methods cannot be directly applied to ICS environments for two main reasons. First, device fingerprinting methods on ICS devices rely on different features from the ones in general computing environments. Instead of using network layer knowledge, they attempt to use behavior on physical devices to reveal device model or types of operations. For example, work in [10] uses the latency between the TCP acknowledge packets and the actual response from the same device to estimate execution time there. Since each vendors may choose different materials or algorithms to implement the same control functionality, this time can be accurate in fingerprinting those devices. Second, existing traffic obfuscation is performed over encrypted channels, which are necessary to mix the obfuscated and normal traffic. Unfortunately, ICS networks still rely on unencrypted traffic due to a wide range of legacy devices, despite ICS network protocols may provide security features. Directly padding or adding dummy packets can easily reveal obfuscated packets, exposing the the underlying traffic pattern to adversaries.

In this paper, we present a framework that schedules when selected packets of an ICS transaction leave the network, so that a timing feature an observer measures becomes a value we choose rather than a property of the device. A programmable switch at the outstation edge holds the acknowledgment and the response of a transaction and releases each at a deadline it computes when the transaction begins.

Because the switch forwards the same packets it received, the endpoints run unmodified firmware and no DNP3 byte is altered, which is what the plaintext setting requires. The framework is general over which packets are held and what the released interval should be, and we demonstrate it on two

network-observable intervals: the cross-layer response time of a poll, the feature Formby et al. [10] fingerprint, and the master-visible timing of a select-before-operate control command. The second is our own case rather than their second feature, which is the device’s physical operation time and not an interval read from the network. These are two case studies of one mechanism, not two systems.

A. Contributions

To overcome the two obstacles above, we design a timing-obfuscation framework. It replaces a selected within-transaction timing feature with a configured value by scheduling the release of real protocol packets. We derive the coupling between the acknowledgment hold, the response hold and the configured interval that any such schedule must satisfy (Section IV).

Realization. We realize the framework on an Intel Tofino-1 using paired blocker and hold queues that sustain a hold until a deadline expires, with a read lane timed from the outstation’s acknowledgment and a control lane timed from the request. Forwarding stays open when a response arrives after its scheduled release (Section V).

Evaluation. We evaluate the realization against a physical SEL-751A relay. The report covers how closely the released interval reaches its configured value, what the policy can and cannot cover, the latency the mechanism adds, the hardware resources it uses, and what an adaptive attacker still learns from the features that remain (Section VI).

II. BACKGROUND AND MOTIVATION

The DNP3 read and control transactions. DNP3 carries supervisory control and data acquisition traffic over TCP between a master and an outstation [11]. The master polls the outstation with a READ request, function code 1, and the outstation returns the requested measurements in an application-layer response, function code 0x81. A control command instead uses select-before-operate. The master sends a SELECT, function code 3, that names an output point, and the outstation returns a SELECT response. The master then sends an OPERATE, function code 4, which the outstation confirms with a solicited OPERATE response.

Because DNP3 runs over TCP, the outstation’s transport layer acknowledges each request, and on the relay we evaluate it does so in a segment of its own before the application answers. TCP does not require that: an acknowledgment may be carried on the response instead [12]. Here the two are separate, so each transaction appears on the wire as a request, a transport acknowledgment carrying no DNP3 payload, and a DNP3 response, in that order, as Figure 1 shows, and we use those three words for those three packets throughout. This separation also becomes the mechanism’s eligibility condition, since the mechanism applies where the acknowledgment and the response are distinct packets and forwards the exchange unchanged where they are not.

Why the interval between them is a fingerprint. The two answers come from different parts of the outstation. Its TCP

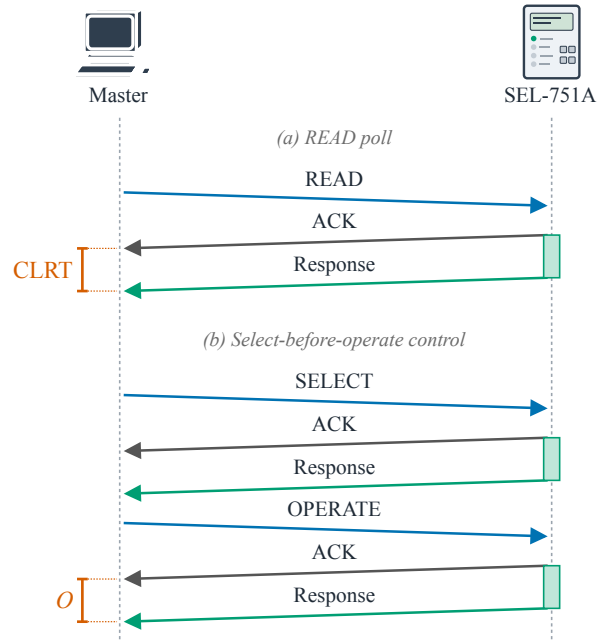


Fig. 1. The two DNP3 transactions we measure, on the master-facing link. (a) A READ poll, whose cross-layer response time runs from the acknowledgment to the response. (b) A select-before-operate control, whose master-visible interval runs from the OPERATE’s acknowledgment to its solicited response. The shaded bar marks the outstation producing the answer; the CLRT and O are measured at the master.

stack emits the acknowledgment as soon as the request arrives, whereas its application emits the response only after the device has done the work the request asks for. Consequently the interval between them, the cross-layer response time or CLRT, becomes dominated by the outstation’s own processing. Since the two packets traverse the same path in the same direction, the common propagation cancels, leaving the device’s work plus whatever differential queueing the path adds between them. This time can differ between device models and between the kinds of work a device is asked to do. On our relay a control command takes 0.8 ms longer to answer than a poll, 2.937 ms against 2.116 ms, while the two requests are acknowledged 0.003 ms apart. Formby et al. used exactly this interval to tell device types and models apart in a substation network [10], and our own measurements show the same separation on a single device, which Section VI reports.

Since the DNP3 function codes travel in plaintext and already name each operation, what timing adds is a second, device-derived signature of that operation.

What a programmable switch makes possible. A programmable switch exposes a match-action pipeline, written in P4 [13], that parses headers, keeps state in registers, and places packets into output queues at line rate. Since such a pipeline can delay a packet and release it later without a host in the loop, a switch between the master and the outstation can hold the acknowledgment and the response of a transaction and release them on a schedule an operator sets, changing neither endpoint nor any byte of the payload.

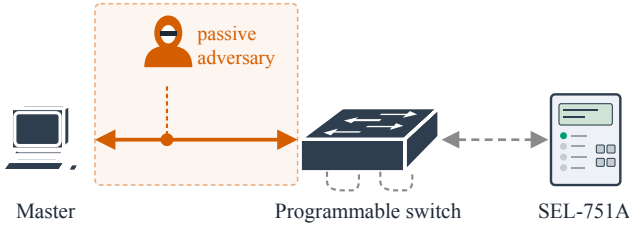


Fig. 2. Observation model. The adversary sees only the master-facing link (shaded); the relay-facing link and the switch-internal lanes (dashed) are observed by neither the adversary nor us.

III. THREAT MODEL AND OBJECTIVES

The adversary. We assume a passive observer on the master-facing link, the adversary of the fingerprinting attack we defend against [10], sitting where Figure 2 shows. It records TCP and IP headers, segment sizes and arrival times. It does not modify or inject packets, compromise either endpoint, control the switch or read its internal state, and it sees neither the relay-facing link, which in our deployment is inside the switch with no tap on it, nor physical breaker motion. Because DNP3 carries no transport security by default we assume the traffic is unencrypted [14], so the observer also reads the DNP3 function codes directly. An adversary that probes or injects remains out of scope, since the traffic-obfuscation work we build on targets the passive observer as well.

What the adversary is looking for. Reconnaissance precedes the attacks that matter here: false data injection requires the attacker to know which devices it is dealing with [15], and fingerprinting supplies that knowledge without the payload, since the timing of the exchange carries it.

Following Formby et al. [10], two intervals fall in scope. The first, the CLRT of a READ or of the SELECT phase of a control, measures the outstation’s own processing and carries the separation between the classes that Section II shows. The second spans the master-visible interval between an OPERATE’s acknowledgment and its solicited response. The adversary also sees the request-to-acknowledgment interval, whose median reaches 0.56 ms on this device. Since the mechanism does not target that interval, we report it as measured.

Our own observation remains narrower than the adversary’s, since every quantity we measure derives from three packet timestamps on the master-facing link, the request, the acknowledgment and the response. We captured neither the switch’s release instants nor any event on the relay-facing link, so where the paper names them they come from the configuration.

Formby et al. estimate the time a device physically takes to complete an operation from a sequence-of-events timestamp inside the outstation’s application payload, and their alternative estimate from packet arrival times produced no usable result [10]. We measure a packet-timing feature of the solicited control response: not physical actuation time, which we do not measure anywhere, and not the payload-timestamp feature, which a mechanism that moves packets

without editing bytes could not affect. The narrower interval still merits defending, because it separates the two classes on its own and Section VI-F shows an adversary recovering 0.782 balanced accuracy from exactly that kind of difference. RO2 therefore asks whether the interval becomes a policy value rather than whether the mechanism conceals physical timing.

What our evaluation actually separates. Our testbed holds one outstation, so no second device exists to confuse it with and no device identification to measure. We measure whether the timing of an exchange still reveals *which class of transaction* it was on that one device, a three-class problem over READ, SELECT and OPERATE with a chance level of 0.333 balanced accuracy. A result here suppresses a class-conditioned timing feature; it does not show that two devices become indistinguishable.

Two conditions, not two adversaries. The *fixed* condition learns transaction classes from undefended traffic and applies that model unchanged once the defense is deployed, which matches the adversary the reconnaissance setting describes. The *adaptive* condition collects obfuscated traffic and retrains. It answers what remains learnable once the defense is deployed, and it recovers 0.782 balanced accuracy where the fixed one stays at chance. It needs no access the fixed one lacks, since the same passive observer sees obfuscated traffic and the plaintext function codes label it. Both conditions are ours, chosen to bracket the problem, and the second limits our claim.

Research objectives. We state the goal as five properties, which the evaluation answers in turn.

- **RO1 (read timing).** The visible CLRT of READ and of the SELECT phase of SBO is a configurable policy value rather than the outstation’s own processing signature.
- **RO2 (control timing).** The master-visible OPERATE response-to-ACK interval stays at a policy value while the relay-facing hold is drawn from a configured codebook of 2, 6 and 12 ms.
- **RO3 (timing leakage).** The timing features carry less information about the transaction class, and a fixed adversary that trained on undefended traffic is reduced to approximately three-class chance, which for READ, SELECT and OPERATE means 0.333 balanced accuracy.
- **RO4 (release policy and coverage).** The release budget, the range of policies the mechanism can realize, the fraction of traffic it can cover, and the residual it leaves are characterized rather than assumed.
- **RO5 (cost and protocol preservation).** We measure the latency the mechanism costs and the frames and bytes it adds, and the external DNP3 workload still completes: every request is answered and every control operation returns a success status.

What is out of scope. Two boundaries follow from the model above and hold for every result we report. The framework does not hide the DNP3 function codes, which stay in plaintext, so RO3 concerns the timing channel and not the command semantics. It also forwards every packet at its original size, so it adds no size or segmentation signature of its

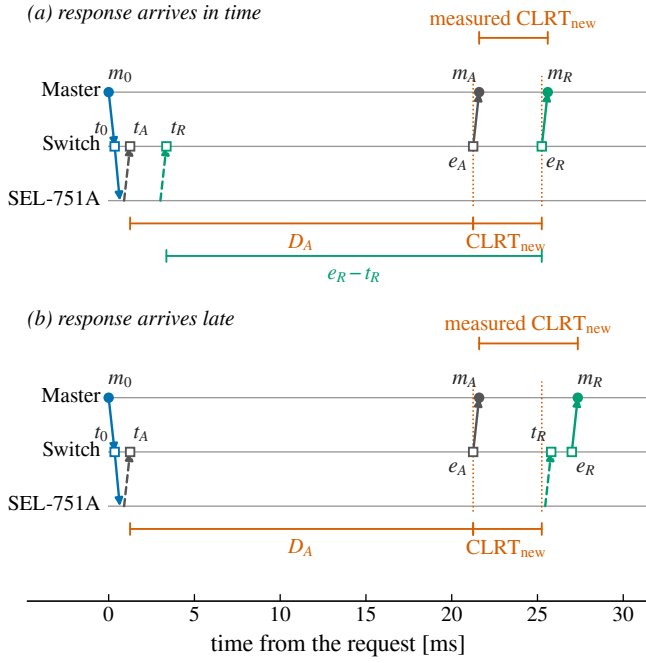


Fig. 3. Release timeline of one READ transaction. Time coordinates are illustrative, chosen so an arrival and its emission are separable on the page. Both deadlines are computed from the acknowledgment's arrival t_A : in (a) the response arrives in time and the master observes the configured CLRT_{new} , in (b) it arrives late. The response hold $e_R - t_R$ is drawn in (a) because the schedule implies it rather than configuring it. Filled circles lie on the master-facing link, the only place this work captured anything; open squares are switch-side instants that were not captured.

own, while the delays it imposes are themselves observable: an adversary who can time packets can see that something is holding them. Because the relay-facing link is not instrumented, as stated above, the per-transaction hold drawn from the configured codebook of 2, 6 and 12 ms is configured rather than observed, which is why RO2 speaks to the master-visible interval alone.

IV. DESIGN

The objectives of Section III require a selected interval to become a value the operator sets. We derive that from one transaction and defer the hardware realization to Section V.

A. A timing model for one transaction

Consider one READ transaction, drawn in Figure 3. The master sends a request. The outstation answers it twice: first its transport layer acknowledges the request, and later its application layer returns the response with the data. We write t_A for the instant the acknowledgment reaches the switch and t_R for the instant the response reaches it. The switch forwards each packet onward at some later instant, e_A and e_R respectively. These are *actual* departures, and we write $\hat{e}_A$ and $\hat{e}_R$ for the instants the schedule asks for, because the two differ by whatever the switch takes to act on a deadline and that difference is part of what we report. The master

therefore observes the interval $e_R - e_A$, up to the propagation and capture delay of the two packets on the link between them.

That interval carries the information about the device, and on our relay it has a median of 2.116 ms for a poll and a spread of 2.6 ms:

$$\text{CLRT}_{\text{original}} = t_R - t_A, \quad (1)$$

the interval between the two packets arriving at the switch. This interval remains dominated by the time the outstation spends between acknowledging the request and producing the answer, but it does not measure that time cleanly, since the two packets traverse the relay-facing link separately, allowing differential queueing and the outstation's own transmission to enter it. Section II explains why it identifies the device; here it becomes the quantity we intend to remove.

To change what the master sees, the switch holds each packet. We write

$$D_A = e_A - t_A \quad (2)$$

for the *acknowledgment hold*. Since it is written from actual departures, a configured D_A and the hold a transaction realizes become the same quantity only to the extent that the release follows its deadline. We refer to $e_R - t_R$ as the *response hold* without giving it a symbol of its own, because the schedule below fixes it rather than letting us choose it. Substituting into the interval the master observes gives the identity that governs the whole design,

$$\text{CLRT}_{\text{new}} = e_R - e_A = \text{CLRT}_{\text{original}} + (e_R - t_R) - D_A. \quad (3)$$

Equation (3) makes the difference between two kinds of defense concrete. If both holds stay constant, their difference stays constant too and the observed interval becomes the original one moved by it: the distribution slides along the axis and keeps its width, a spread of 2.6 ms wherever it lands, so an observer who has seen the device before still recognizes it by the shape. The response hold must instead absorb the variation in $\text{CLRT}_{\text{original}}$, which we achieve by computing both releases from the same instant t_A . When the acknowledgment arrives, the switch computes both release instants immediately,

$$\hat{e}_A = t_A + D_A, \quad \hat{e}_R = t_A + D_A + \text{CLRT}_{\text{new}}, \quad (4)$$

where CLRT_{new} is the interval we want the master to observe. A packet's actual departure e follows its scheduled instant $\hat{e}$ by the time the blocker queue takes to drain and by the queue's own service, which Section V describes and Section VI-G measures on an instrumented build. Writing the two apart keeps the identity honest: (3) holds exactly for actual departures, while (4) speaks to the schedule rather than to the wire. We call this value the *configured* CLRT_{new} , and we call what the master actually records the *measured* CLRT_{new} ; Section VI reports how far apart the two are. Because both release instants derive from t_A and neither from t_R , the term $\text{CLRT}_{\text{original}}$ cancels in (3) and the observed interval becomes the configured one. Consequently the response hold is implied rather than chosen,

$$e_R - t_R = D_A + \text{CLRT}_{\text{new}} - \text{CLRT}_{\text{original}}. \quad (5)$$

Equation (5) explains why the three quantities cannot be picked independently. Once the acknowledgment hold and the configured CLRT_{new} are fixed, the response hold follows from whatever the device did on that transaction. It varies from one transaction to the next exactly as much as the device does.

This holds only while the response exists in time to be released on schedule. The switch can delay a packet but cannot emit one it has not received, so (4) requires

$$t_R \leq t_A + D_A + \text{CLRT}_{\text{new}}. \quad (6)$$

The acknowledgment may leave before the response arrives; that ordering is normal and does not by itself imply a lost packet. When (6) fails the deadline is already past when the response appears, so the switch marks it late and commits it to the response-hold queue, from which it leaves when that queue next serves it, as in Figure 3(b). Consequently the master usually observes an interval larger than the configured value, though not necessarily, since what it measures remains a difference of two releases, each carrying its own queueing and service, so a late response alone does not guarantee a larger measured interval. One read-lane exchange in a thousand answers too late to be scheduled, and we include those exchanges in every figure and table below, since they mark the boundary of the mechanism rather than a defect of the measurement.

The holds decide what the master observes *between* the two packets, not how long it waits for the answer itself, which is the quantity an operational requirement constrains. Write D_R for that: the response's whole journey from the outstation transmitting it to the master receiving it. Since only the master-facing link is instrumented, we do not measure D_R , nor the portion of it from the switch onward: m_R is captured while t_R , the response's arrival at the switch, is not. The quantity

$$m_R - t_R \leq D_R \quad (7)$$

is therefore a modelled partial journey, written from the model's endpoints rather than from timestamps we hold, and full D_R remains unmeasured.

Two terms make up the observed portion, $(e_R - t_R) + (m_R - e_R)$: the time the switch holds the response and the path from the switch to the master. The hold itself divides once more, into the wait to the release deadline, the interval ε from that deadline to the end of blocking, because the switch stops blocking a queued packet only once the queue gating it has drained, and the service that follows. Those three sum to the hold rather than adding to it. Consequently neither the response hold nor the configured CLRT_{new} is the response latency, and the three are not separately selectable: fixing D_A and the configured interval fixes the hold, which in turn consumes part of whatever the application deadline allows. Section VI-G reports what we measured of ε and on which build.

B. Choosing the holds and the configured interval

The model leaves three quantities to decide: the acknowledgment hold D_A , the configured CLRT_{new} , and, through (5),

the response hold that follows from them. A different constraint limits each of them.

The transport limits the acknowledgment hold in principle. Holding an acknowledgment delays the feedback the master's TCP uses to decide that a segment is lost. Consequently, a hold approaching the master's retransmission timeout would make the master resend a request the outstation had already answered.

That timeout has to be measured, not assumed. RFC 6298 [16] recommends rounding it up to a second, but that recommendation concerns a minimum and names no property every stack exhibits: on our master an unacknowledged request repeats after 201 ms. The 20 ms hold we evaluate consumes about a tenth of that, and a hold of a few hundred milliseconds would begin to race it. We do not argue that the timer's adaptivity makes a constant hold safe, because adaptation says nothing about the first delayed exchange, a reconnection where the timer restarts, or a policy change mid-session.

In practice the transport bound does not bind. The mechanism sustains a hold with a budget on blocker passes rather than a wall clock. From that budget, an assumed service rate, a conservative bound on the outstation's acknowledgment latency, measured detection and release terms and a safety margin, the control plane computes a horizon H of 30.8 ms and an admissible budget of 24.8 ms. Three limits act here, and they stay distinct. The horizon carries advisory force and reports what the admission arithmetic computes. A separate clamp refuses any acknowledgment hold above 40 ms outright. The sweep of Section VI-D deliberately installs settings past the horizon, to find where the mechanism leaves its operating region. None of the three names a deadline the data plane enforces. The distinction matters because a pass budget bounds how many passes get *served*: under strict priority a lower-priority reservoir can be starved while a higher-priority one is still being served, so one budget does not translate into one elapsed time for every queue. That sweep shows the consequence directly. At a configured acknowledgment hold of 36 ms the median request-to-acknowledgment interval saturates at 31.07 ms, well short of the configured value. The median request-to-response interval for the same point is 40.58 ms and the median CLRT reaches 9.51 ms rather than the configured 4 ms. The acknowledgment reaches a ceiling; the response does not stop at it. We observed that saturation as a transition on the acknowledgment lane at this configuration. It names no release deadline the hardware guarantees for every held packet.

The operation limits the configured CLRT_{new} , and through D_R rather than directly: an operator accepts a bound on how long the answer takes to arrive, not on the gap between the two packets that carry it. We could not verify a communications requirement that fixes such a bound for a DNP3 poll or a select-before-operate control on a distribution relay. The only numeric deadline we could verify from primary documentation is the relay's own select-to-operate timeout, documented default 1.0 s [17]. It governs how long a select stays armed, not

how quickly a response must return. Timing classes defined for other protocols do not transfer to a DNP3 poll merely because their numbers are convenient.

We therefore treat the setting as a deployment choice, not a compliance question. We state the value we tested, 4 ms, and report the response latency the mechanism produces, about 23 ms more than the device alone, so an operator with a requirement of their own can check it against those numbers.

A third timer is easy to miss. Holding the response also postpones the master's acknowledgment of it, which is the feedback the *outstation's* transport waits for, so the response hold spends the outstation's retransmission budget as well as the application's patience. The budget belongs to the outstation, not the master. On this relay its timer runs about three orders larger than the master's, so the master's timer binds here. The three constraints stay separate, and a deployment that changed either device could reorder them.

Together the two set what the mechanism costs. Their sum

$$D = D_A + \text{CLRT}_{\text{new}} \quad (8)$$

is the release budget: by (6) it decides which transactions the switch can place on the schedule at all. A larger budget covers more of the device's own response-time distribution and costs more delay on every transaction it covers: at $D = 24$ ms it covers 99.9% of them for about 23 ms of added latency. D names the budget rather than the delay added to each exchange. The switch holds a transaction the outstation answers quickly for longer than one it answers slowly, so the added delay reaches at most D without equalling it.

Because the budget and the observable measure different quantities, an operator can fix the cost and still choose the feature. Holding D at 24 ms, we installed configured values from 1 ms to 22 ms. The master observed each one while the added delay stayed the same. Section VI-D reports that sweep and the point at which the realized hold stops tracking the requested one.

The model says nothing about what CLRT_{new} should be, so the framework and the policy it carries stay separate. We chose a constant, because it removes the device's variation entirely and an operator can state and check it in one line. A schedule could instead draw the value from a distribution or compute it from the operation, and the same mechanism would carry it; we implement and evaluate only the constant setting.

C. The control transaction is timed from the request

A select-before-operate control command differs in one way that matters here: the framework holds the command itself before it reaches the relay, separating the moment the master issued the command from the moment the relay acts on it. The outstation's acknowledgment cannot serve as the reference instant, because it does not exist until after the command has been released, and any deadline measured from it would carry the length of the command hold into the interval the master observes. We therefore time the control lane from the request. When the OPERATE arrives at T_0 , the switch releases

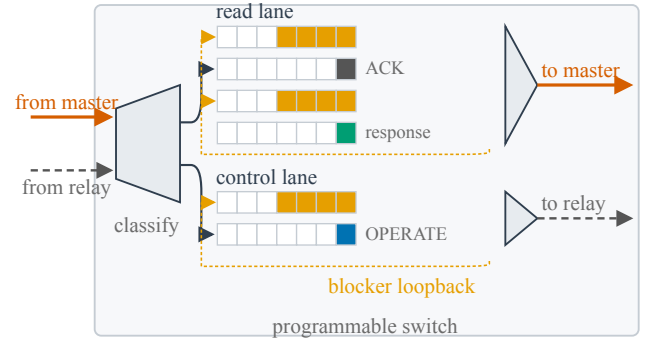


Fig. 4. How one pipeline realizes the schedule. Ingress classifies each transaction and computes its release deadlines. Amber queues carry generated blocker packets, drained ahead of the held packet to sustain a hold until its deadline expires; each lane recirculates its own on a separate loopback port, in a scheduling domain of its own, so the read reservoirs cannot starve the control one. The four-queue ladder holds what the outstation returns, for READ and for both phases of a control alike; the second pair holds the outgoing OPERATE, not a second reply. Solid lines are master-facing and observed; dashed lines are not, as in Figure 2.

it toward the relay after an internal delay J , and places the acknowledgment and the solicited response the master sees at

$$e_{\text{ack}} = T_0 + D_A, \quad e_{\text{or}} = e_{\text{ack}} + \text{CLRT}_{\text{new}}. \quad (9)$$

Those two offsets are the read lane's own, not a second pair: one set of deadline registers serves both lanes (Section V). The master therefore observes $O = e_{\text{or}} - e_{\text{ack}} = \text{CLRT}_{\text{new}}$, in which J does not appear, so the relay-facing release time can vary while the master-facing interval does not, and an observer cannot subtract a known offset to recover it.

The control plane keeps its own pair of offsets for this lane and refuses one whose acknowledgment offset does not exceed the largest admissible J plus the outstation's acknowledgment latency, or whose response offset reaches the horizon. Those checks decide what may be installed, not when a packet leaves. Our experiments set D_A to 20 ms and the configured CLRT_{new} to 4 ms, so both lanes present the same value to the observer.

D. What the design does not change

The mechanism replaces one interval per lane and leaves the rest of the exchange untouched, within the scope Section III sets out. The schedule causes one consequence directly. The request-to-acknowledgment interval still carries the outstation's own acknowledgment latency, displaced by the constant D_A but not reshaped by it. Section VI-F reports what an attacker allowed to retrain recovers from it.

V. IMPLEMENTATION

Section IV says when each packet should leave, but a forwarding pipeline has no way to wait: a P4 program runs to completion per packet and can read a clock, yet it cannot sleep or schedule a callback.

A. Holding a packet with queues

The switch does carry a traffic manager with several queues per port and a strict-priority scheduler, the primitive with which SP-PIFO approximates programmable scheduling on today's hardware [18]. Since a scheduler decides what does *not* leave, we hold a packet by using that property rather than a timer.

A READ uses *four* queues on one internal port, as Figure 4 shows. Two carry the packets we delay, the acknowledgment and the response, and above each of those sits a *blocker queue* in strict priority carrying small packets the switch generates itself. While blockers keep arriving, the packet beneath them is never served, and each hold ends when its own blocker queue runs dry. The response does not queue behind the acknowledgment, having its own held queue and its own blocker gate, which lets the switch schedule the two releases independently. In descending priority the ladder runs acknowledgment blockers, held acknowledgment, response blockers, held response. Blockers circulate on an internal loopback port and never reach a cable, so neither endpoint can observe them. Each carries a tag naming its transaction and lane, so a leftover blocker cannot delay a later transaction.

No count drives this timer. Instead the program stores an absolute release instant when the acknowledgment arrives, quantized to a 256 ns grid, and each of the 64 recirculating blockers compares the current timestamp against it: while the difference stays negative the program regenerates the blocker, and once it turns, the program drops it. Since the deadline decides when the packet leaves, the blockers only have to keep the queue non-empty until then, so the reservoir becomes sized for continuity rather than for duration.

The queue does not empty at the deadline, however. Since each blocker tests the deadline on its own pass, those still queued when it passes are served once more before the queue runs dry, allowing the held packet to leave. The release therefore trails the deadline by the time that draining takes, and each lane drains its own queue, the acknowledgment's above the response's; Section VI-G reports what we measured of that interval. The arrangement makes the coupling of Section IV-A implementable: the program computes the response's release instant at the acknowledgment, so the hold absorbs whatever the outstation does without the switch measuring it.

B. One READ transaction

The request arrives, a table classifies it by function code, and the program forwards it at once. It records a transaction in flight and the acknowledgment to expect. When the acknowledgment arrives at t_A , a median of 0.56 ms after the request on our relay, the program writes both release instants from that one timestamp, which places them on the same instant t_A that Section IV-A requires. It does so once from an unarmed sentinel so a duplicate acknowledgment cannot arm a second deadline. The acknowledgment enters the hold queue and the packet generator fills the blocker queue. A response that arrives before its deadline queues behind the acknowledgment, and the two leave in order at their scheduled instants. A response that

arrives after its deadline has no remaining wait to serve. The program marks it late and commits it to the same response-hold queue rather than to a bypass, as in Figure 3(b). It leaves when that queue next serves it, so its departure is prompt but not instantaneous. Releasing the response clears the transaction state.

The two lanes hold different packets. The four-queue ladder above holds what the outstation sends back, the acknowledgment and the response, for READ and for both phases of a control alike. The second loopback port carries a pair of queues for the opposite direction: blockers above a held *OPERATE request*, on its way to the relay. So the control extension adds a hold the read path does not have, while the same machinery releases the packets the master observes in both cases. Giving the OPERATE its own scheduling domain keeps the higher-priority read reservoirs from starving it.

The queues stay separate; the transaction state does not. All lanes share the deadline registers. Every register that records a transaction holds a single slot, and the control path writes the same release instants at OPERATE admission that the read path uses, so the program protects one transaction at a time rather than one per flow.

The evaluated configuration satisfies that assumption by construction: the driver serialises its exchanges and every capture carries one connection. Where a second transaction would arm while one runs live, the program does not interleave them silently; it classifies the arming attempt as busy and forwards the packet unprotected rather than taking the deadline already in progress. The claim covers the deadline, not the rest of the transaction's state. The program writes the expected relay sequence and acknowledgment earlier in the same pass than it reaches the busy verdict, so we claim it for the serialized workload we evaluated and not in general. Per-flow protection would need per-flow state and a way to index it, which this program lacks and we did not evaluate.

C. Failing open

Almost every path through the program ends in a forwarding decision. A packet outside the protected session, an unrecognized request, an acknowledgment with no armed transaction and a response whose transaction has retired all leave unchanged, so such an exchange loses its protection and still arrives. One path behaves differently. The control lane keeps a released OPERATE's application generation as a spent marker until the next SELECT clears it, and the switch drops a later OPERATE carrying that generation. A retransmission the master sends to repair an OPERATE lost after the switch forwarded it carries exactly that generation. Consequently the switch discards the packet that would have repaired the loss (Section VI-G). A budget on blocker passes bounds every hold: once a reservoir has spent its budget, the packet leaves and its state clears. The budget counts passes rather than elapsed time, so the horizon H of Section IV-B, 30.8 ms here, records what the control plane computes from the budget and an assumed service rate rather than a wall-clock deadline the data plane keeps. With the timing mode off the same program forwards

everything immediately, which is how we collected the Timing OFF arm.

D. Resources and testbed

The program runs to about 1,560 lines of P4₁₆ [13] for the Tofino Native Architecture, compiled with the Barefoot SDE 9.13. It occupies all twelve ingress match-action stages of one pipeline and six egress stages, with 112 tables. State is small: one-entry registers reached by stateful ALU actions, one access per packet per register, the most the architecture permits. One 25 Gb/s port carries the master and one 1 Gb/s port the relay. Two further ports serve as loopbacks carrying no cable: the first carries the four-queue read ladder, the second the control extension's pair, blockers above a held OPERATE. The second loopback sits in its own scheduling domain, so the higher-priority read reservoirs cannot starve the OPERATE reservoir. Six queues across two loopbacks in all.

Every policy quantity lives in a runtime table entry, so an operator changes policy by writing entries rather than loading a program. The loaded build realizes the dual-deadline schedule only, because its arming table admits that mode alone. The acknowledgment-only and response-only limits of the model therefore describe the design rather than settings of this build: configuring them left both packets unheld, with the measured interval the outstation's own.

The control plane, about 890 lines of Python, brings up ports and queues, installs the packet-generator configuration, and writes the timing parameters and reads them back. It refuses through a clamp, not through the admission arithmetic. It rejects a request for an acknowledgment hold above 40 ms outright before any write, which we confirmed on the switch, and the acknowledgment offset A on the control lane must exceed the largest admissible J plus a bound on the outstation's acknowledgment latency. It computes and reports the horizon H of Section IV-B without enforcing it, which is exactly why the sweep of Section VI-D can install settings beyond it and find where the mechanism leaves its operating region. The outstation, an SEL-751A feeder protection relay, belongs to one of the DNP3 device classes whose timing Formby et al. fingerprint [10]. The master issues READ and select-before-operate operations to isolated control points through a guarded driver, so no control operation in the evaluation moves a breaker.

VI. EVALUATION

A. What we measured, and where

Evaluation Cases. We compare two arms of the same program. They differ only in the release schedule. In *Timing OFF* the switch forwards the outstation's acknowledgment and response as they arrive; in *Obfuscated* it holds them and releases them on the deadlines of Section IV. The switch is the only path between the master and the relay, and every timestamp is taken on the master-facing link. Consequently each interval below is one a passive observer can record: the arrival of the transport acknowledgment to the arrival of the application response, both on the master's own interface. The

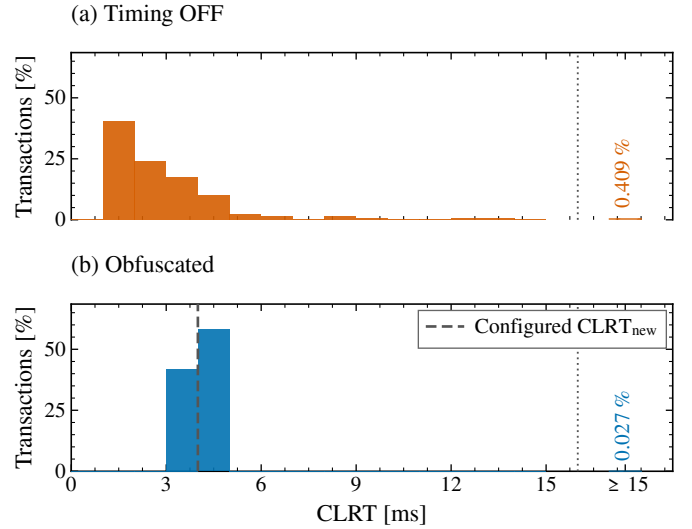


Fig. 5. Measured READ CLRT before and after obfuscation, as the percentage of each arm's own transactions per bin, on common 1 ms bins anchored at 0 ms with a linear ordinate. Finite bins are half-open; the category past the break marker holds everything at or above 15 ms, 0.409% of (a) and 0.027% of (b), written beside it. The dashed line in (b) is the configured $CLRT_{new}$, which falls on a bin edge, so the obfuscated mass divides between the two neighbouring bins; Figure 6 resolves it. In (a) $n = 26,400$, mean 2.800 ms, sd 2.609 ms, variance 6.808 ms^2 ; in (b) $n = 26,400$, mean 4.012 ms, sd 0.628 ms, variance 0.394 ms^2 .

switch's internal instants are on links that were not captured and are recovered from the program rather than measured.

Configuration provenance is partial. Each block's manifest records the policy it was configured with and the loaded program's identity, but the sweep has no per-point readback of what the device held. Every policy value in this section is the configured one unless we say otherwise, and we mark where a readback would have settled a question the configuration cannot.

Dataset. We report per DNP3 exchange. The campaign holds 63,360 of them, half with the mechanism running and half without, collected over five hours in interleaved blocks so that drift cannot separate the two arms. Every request was answered, and no exchange was retransmitted, malformed or excluded. A separate sweep on the same testbed covers 16 configured policies and three controls with the mechanism disabled.

B. Does the released interval reach its configured value? (RO1)

RO1. The read-path schedule of (4) does replace $CLRT_{original}$ with the configured $CLRT_{new}$. We compare the two arms over all 22 grouped runs as full empirical distributions, because an adversary reads a single observation against the shape of the Timing OFF distribution.

Figure 5 shows the two READ distributions. Before obfuscation the interval spans 1.01 to 83.46 ms and stays multi-modal, with peaks near 1.2, 2.1 and 4.0 ms; after it, 99.9% of transactions fall within 0.10 ms of the configured $CLRT_{new}$. In Figure 6 the released interval resolves into a

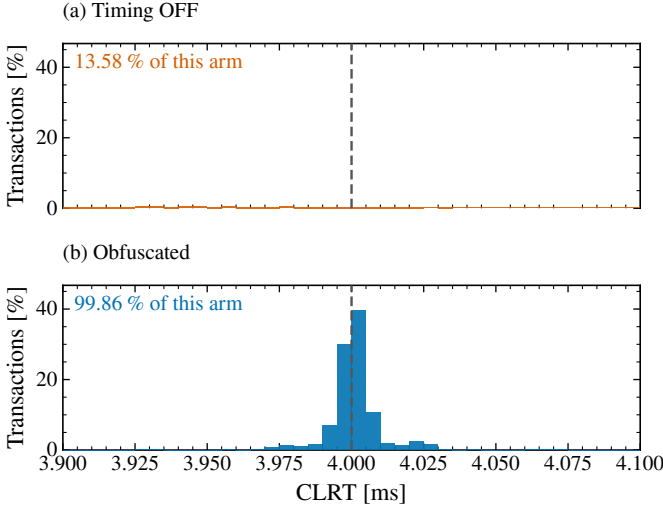


Fig. 6. The released interval, resolved. The same measurements as Figure 5 on 0.005 ms bins within 0.10 ms of the configured CLRT_{new} (dashed). The ordinate is still the percentage of that arm’s entire sample, not of the window, so the bars are directly comparable with the other figures. The window holds 13.58% of (a) and 99.86% of (b).

narrow distribution about that value, where the Timing OFF arm places only 13.6% of its transactions.

The change is a replacement, not a shift. The standard deviation falls from 2.609 to 0.628 ms for READ and from 2.267 to 0.024 ms for SELECT. Under a session-level bootstrap those give variance ratios of 0.058 (95% CI [0.019, 0.101]) and 0.0001 (95% CI [1.4×10^{-5} , 3.4×10^{-4}]). A defense that shifted the interval would keep that spread, since translating a sample leaves its variance unchanged. Translated onto the protected median, the Timing OFF distribution retains standard deviations of 2.609 and 2.267 ms, four and ninety-four times what we measure. We computed that counterfactual analytically, from the Timing OFF samples themselves, and since the loaded program has no shifting mode, the counterfactual stays analytic and we do not present it as a measured arm.

Low spread at the wrong interval would not be normalization, so we also report where it sits. The median $\text{CLRT}_{\text{original}}$ is 2.116 ms for READ and 2.050 ms for SELECT, with interquartile ranges of 2.778 and 2.697 ms; under the mechanism both medians sit at the configured 4.000 ms and each interquartile range falls to 0.006 ms, two orders of magnitude below the 1 ms step between neighbouring policy settings in the sweep of Section VI-D. The median alone understates the change, since the Timing OFF distributions rise in a few discrete steps, allowing one observation to be informative about the class, whereas the obfuscated ones become a step at the scheduled release, an instant computed from the acknowledgment and a fixed offset, which no longer depends on how long the device took. Figure 5 shows that for READ; the per-class distributions for all three, and the per-run medians in acquisition order, are published with the artifact.

A tail remains, and it falls on one side only: the largest obfuscated READ interval reaches 57.182 ms, and fewer than

one READ exchange in a thousand departs from the scheduled release by more than 1 ms. This asymmetry follows from the mechanism, since a response reaching the switch after its scheduled release cannot be moved backwards and leaves as soon as the hold queue serves it, the mechanism being unable to schedule a release before the response exists. Below the configured value the distribution keeps mass as well, since each release carries its own queueing and service, so their difference falls either side of the configured interval, which gives the distribution its width. Those late exchanges are the coverage boundary of RO4 seen from the other side, and Section VI-D counts them.

Stability within the Campaign. The obfuscated median sits at the scheduled release in every one of the 22 grouped runs and every class, while the Timing OFF medians stay separated, so the effect is not an artifact of one run or of drift over the five hours. The per-run medians and interquartile ranges in acquisition order are published with the artifact.

C. Does the control lane behave the same way? (RO2)

RO2. The control-path rule of (9) holds the master-visible response-to-ACK interval at the policy value. Timed from the request, its observable is $O = \text{CLRT}_{\text{new}}$, in which the internal hold J does not appear. Because that deadline is computed from the request, the read-lane coverage of Section VI-D does not describe it, and the switch draws J per transaction from the configured codebook {2, 6, 12} ms. The OPERATE median moves from 2.937 ms under Timing OFF to 4.000 ms under the mechanism, the interquartile range falls from 2.827 to 0.006 ms, and the interval stays concentrated at the configured value in every run.

D. What range of settings does the mechanism support? (RO4)

RO4. The interval a passive observer records becomes a policy value the operator sets, over a usable range. We installed 16 release policies in turn and ran a fixed DNP3 workload through each, in two series: the first holding the budget at $D = D_A + \text{CLRT}_{\text{new}} = 24$ ms and moving the split between them, the second fixing the configured CLRT_{new} at 4 ms and raising D_A until the mechanism leaves its operating region. Each point gives the median over its READ exchanges.

In Figure 7(b) the measured CLRT_{new} follows the configured value along the identity line: settings of 1, 2, 4, 8, 12, 16, 20 and 22 ms give measured medians of 0.998, 1.999, 3.999, 8.001, 12.001, 16.000, 20.001 and 22.001 ms, while the end-to-end response time stays between 25.30 and 25.34 ms throughout. The leaking interval and the cost of the exchange can therefore be set independently, which makes the mechanism a framework rather than one fixed setting. In Figure 7(c) the request-to-acknowledgment interval tracks D_A up to 30 ms, where the measured median reaches 30.571 ms and CLRT_{new} still sits at 4.001 ms.

Beyond that the acknowledgment interval saturates at about 31.07 ms and CLRT_{new} can no longer be held, rising to 5.503, 7.498 and 9.507 ms at D_A of 32, 34 and 36 ms, because the reservoir carries a finite pass budget and a hold ends when

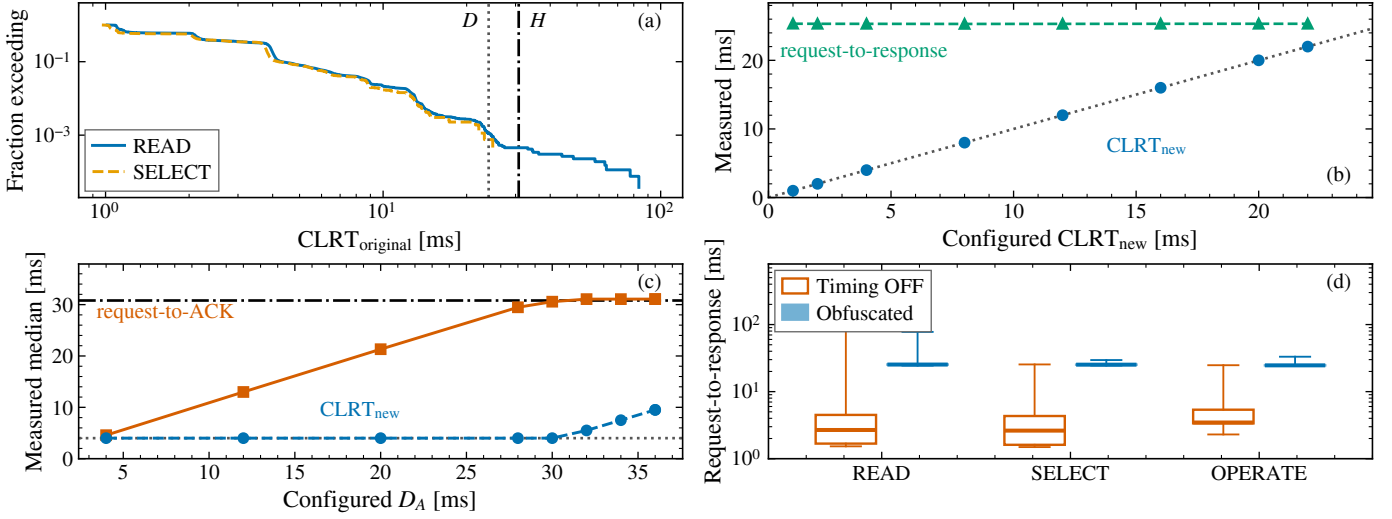


Fig. 7. The release policy is programmable and bounded on both sides. (a) Timing OFF read-lane tail against the budget D and the horizon H . (b) At $D = 24$ ms the measured CLRT_{new} follows the configured CLRT_{new} while the response latency stays fixed. (c) Raising D_A saturates near H . (d) End-to-end response time per class.

that budget is spent. This saturation sits close to the admission horizon H , as one would expect if the control plane’s assumed service rate holds, yet it remains no evidence that the data plane enforces H . From below, the outstation’s own response time closes the range, and the switch cannot anticipate: in Figure 7(a) the 24 ms budget covers 99.900% of Timing OFF read-lane exchanges, leaving one in a thousand arriving too late to be held. Consequently the usable range becomes 1 to 22 ms at a fixed budget.

E. What does the mechanism cost? (RO5)

RO5. In Figure 7(d) the median response time rises by 22.7 ms for READ, 22.6 ms for SELECT and 21.2 ms for OPERATE: close to but below the release budget, because it depends on how long the outstation itself took, and a real operational cost to weigh against the polling period and any protocol timeout. Each exchange carries 3.017 frames and 272.3 bytes on the master-facing link, identical in both arms, so the mechanism moves packets in time without adding a frame or a byte.

Timeout and Retransmission Headroom. Holding the acknowledgment consumes the master’s timers, so we measured what it consumed. On the master-facing captures the request bytes were acknowledged in a separate segment every time, never piggybacked on the response, no request was sent while earlier bytes were unacknowledged, and neither arm carried a retransmission, a reset or a duplicate acknowledgment.

The longest the master waited for its acknowledgment was 29.2 ms and for its response 77.7 ms. Withholding every acknowledgment of one request on a separate diagnostic connection, we observed the master repeat it after 201 ms, about seven times the longest observed wait, consistent with the absence of a retransmission. This 201 ms remains a measured repetition threshold rather than a reading of the sender’s timer, since the capture records packets rather than socket state,

and the next repeat followed 208 ms later, which simple exponential backoff does not explain. The receive timeout in our driver sits at 3 s and bounds one read rather than the whole transaction, so it never acts as a transaction deadline.

Select validity gives the other timer a held control transaction could violate, since the master issues its OPERATE only 0.19 ms after the SELECT response reaches it and the mechanism delays that response: no obfuscated OPERATE exchange was refused for a stale select. All of this characterizes this operating point, and would have to be rechecked for a master with a shorter timeout.

F. What does the attacker still learn? (RO3)

Setup. The classifier sees the two separately measurable intervals; we exclude their sum, the total response time, so that no feature forms a linear combination of the others. Separate measurability does not imply statistical independence and we do not assume it: the classifier receives both intervals and may exploit any dependence, which the adaptive result below does. We estimate mutual information for the CLRT in bits against a null built by shuffling class labels within each grouped run over 1,000 permutations, which preserves the per-run class counts.

RO3. Consider first what the mechanism does to the two intervals before any classifier runs. Under Timing OFF the classes separate mainly along the post-acknowledgment axis, with medians of 2.116, 2.050 and 2.937 ms for READ, SELECT and OPERATE against request-to-acknowledgment medians of 0.555, 0.555 and 0.558 ms, which barely separate them at all. Under the mechanism that axis collapses: all three medians sit at 4.000 ms and READ and SELECT overlap. What remains is a horizontal offset, with the request-to-acknowledgment median moving to 21.336, 21.239 and 20.650 ms, because the read lane takes its origin from the outstation’s acknowledgment and the control lane from the

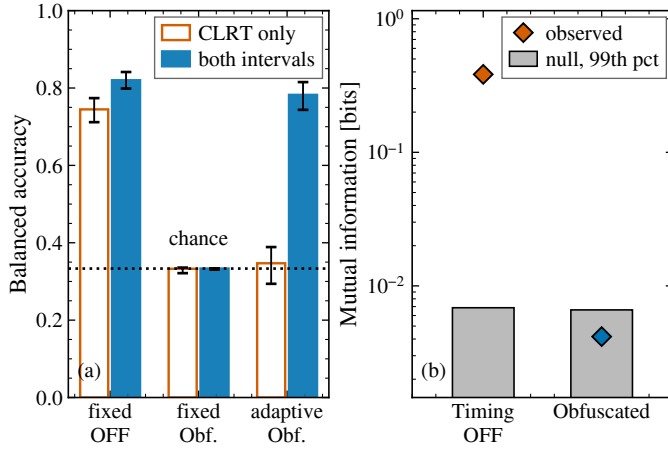


Fig. 8. Transaction-class leakage under the two attacker models, leaving out one grouped run at a time. (a) Balanced accuracy by attacker condition and feature set. The abscissa names the training condition and the arm it is tested on: fixed on Timing OFF, the same fixed model on Obfuscated traffic, and a model retrained on Obfuscated traffic. Whiskers span the 22 held-out runs and are not a confidence interval. (b) Mutual information between the CLRT and the class, against a within-run permutation null.

request. The two intervals plotted against each other, which shows that collapse and that offset directly, are published with the artifact.

In Figure 8 we show the two attacker models. The fixed adversary trained on Timing OFF traffic reaches 0.745 balanced accuracy on held-out Timing OFF exchanges using the CLRT alone, and 0.820 using both intervals, yet applied unchanged to obfuscated traffic it falls to 0.333 in both cases, the chance level. Mutual information between the CLRT and the class falls from 0.383 bits to 0.004 bits, where the Timing OFF value lies far above its permutation null at the resolution of the test ($p = 0.001$ with 1,000 permutations) while the obfuscated value lies inside its null ($p = 0.084$), so the test does not reject the hypothesis of null noise. Failing to reject falls short of evidence for zero, since with 1,000 permutations the test resolves differences of this size only so far, and a small residual dependence would look the same. Consequently, for the evaluated fixed Random-Forest attacker, three-class transaction identification falls to approximately chance, and no dependence between the cross-layer response time and the class survives at the resolution of this test. This holds for the two features we give it: the plaintext function code lies outside a timing-only feature set and still names the operation.

The adaptive adversary gives two results, and they answer different questions. Retrained on obfuscated traffic it reaches 0.347 on the CLRT alone, against a chance level of 0.333 and with a per-run range of 0.294 to 0.389 that straddles it: the targeted feature is suppressed against a retrained adversary, and not against a fixed one alone. The same adversary recovers to 0.782 given both intervals, since the read and control paths are timed from different events, leaving their acknowledgment latencies different. Row-normalised confusion over all 22 held-out runs, published with the artifact, shows the shape of that

recovery: OPERATE separable while SELECT stays confused with READ.

We state the cost of that difference rather than leave it to be inferred. On the request-to-acknowledgment interval alone, balanced accuracy rises from 0.479 under Timing OFF to 0.809 under the mechanism: weakly informative about the class before the mechanism runs, and the most informative single feature afterwards. The mechanism leaves that feature in place, and it creates it, because holding the acknowledgment for D_A displaces an interval the two lanes compute from different events. Against an adversary using both intervals and allowed to retrain, the net effect falls from 0.820 to 0.782, or 0.038 on a three-class problem with a chance level of 0.333. That margin stays small, and we report it as measured.

Notably the mechanism suppresses the targeted feature under both conditions, the cross-layer response time alone leaving the fixed adversary at 0.333 and the retrained one at 0.347. What retraining recovers, it recovers from the request-to-acknowledgment interval, which the mechanism does not target and does make more informative, and that residual bounds the result. Formby et al. [10] profile devices from undefended traffic, which motivates the fixed condition; their setting does not restrict an observer of this mechanism to an unchanged model, and we do not read it as doing so. Timing both paths from the same event would close the residual, and we leave that to future work. We have not searched the space of defenses that might close it differently.

G. Limitations

Each limitation below bounds one of the results above, and we name which.

One Device, One Campaign. The evidence comes from one SEL-751A behind one Tofino-1 in one five-hour campaign, and since the runs remain grouped collections rather than independent replications, every result from RO1 to RO5 becomes a statement about this device and this window, with every spread reflecting within-campaign variability.

What RO3 Establishes. RO3 characterizes the Random-Forest attacker we evaluated rather than every classifier, and the adaptive adversary recovers 0.782 balanced accuracy from the difference between the two events, which bounds what it achieves.

What RO2 Rests On. We captured only the master-facing link, so the realized per-transaction hold J and the relay-facing release at $T_0 + J$ remain configured rather than observed: RO2 claims only the interval the master sees and reports no result per J .

What RO4 Observes. Since the admission horizon H is computed in the control plane rather than enforced by the data plane, the saturation at 31.07 ms becomes an observed transition rather than a proof that the two coincide, and the sweep offsets likewise come from the configuration table rather than a per-point readback.

The Residual Is Not Attributed. The switch’s own release instants were not captured, and the program cannot supply them, since the timestamp registers it declares are never

written. Consequently what the measured distribution shows around the configured value becomes a net difference between two releases rather than a delay we can place inside the switch.

A separately instrumented build adds two registers per lane, keyed on the outcome that marks a blocker terminating because its deadline passed. Over twelve transactions the medians are 1,706 ns on the acknowledgment lane and 1,705 ns on the response lane. The standard deviations are 8.7 and 8.0 ns, about 4×10^{-4} of the 4 ms configured interval. The first blocker’s ingress timestamp follows the deadline by a median of 12.5 and 13.5 ns, bounding when a blocker first observed the expiry rather than when the release decision was taken, which the program does not timestamp.

Four limits apply, and the first is why we avoid the name ε for it. Since both timestamps are taken in ingress on a recirculating blocker token, the interval ends at the last blocking action rather than at the wire, and this program carries no egress timestamp. We report an internal blocker-termination interval and do not claim it bounds ε in either direction, since the blocker’s last traffic-manager service precedes its return to ingress and the ordering of the two endpoints remains unestablished. It also comes from a *different build* than the campaign, characterizing the mechanism rather than any exchange in the measured distributions. The recorded patch over the frozen program identifies that build, and the build tree, its loader configuration and the loader’s log agree on which binary ran. Twelve transactions at one configuration are not a drain bound over load. Comparing the instrumented build against the frozen one over 200 polls each gives mean cross-layer response times of 4.000282 and 3.999470 ms with sample standard deviations of 0.0153 ms in each: one pair of captures at one configuration, which does not establish equivalence, and which would not reveal equal movement in both releases.

What RO5 Counts. The overhead of RO5 is measured on the master-facing link, where the switch’s internal blocker traffic never appears. The zero-retransmission result is bounded the same way: because the program suppresses a response retransmission that matches an already-seen transport position while its transaction is live, such a copy would be absorbed inside the switch and would not reach a master-facing capture.

We probed that suppression. A response lost downstream of the switch is recovered by exactly such a retransmission, so a mechanism that discarded it would break loss recovery. Dropping the response inbound at the master, scoped to one connection, leaves it unacknowledged and the outstation retransmits on its own timer; three further copies crossed the switch and reached the master’s interface, at 3.20, 9.21 and 20.25 s, every one forwarded rather than absorbed.

The probe does not show that duplicate suppression preserves loss recovery. The filter dropped every copy for the whole test rather than only the original, so nothing was acknowledged and the application recovered nothing, and the capture is taken ahead of the filter, so it records arrival at the interface. The copies also arrived long after the transaction had retired, the first 2.97 s after a transaction live for about 24 ms, so they met the path a stale token takes rather than the

path a duplicate of a live transaction takes, which is the case the concern is about. Reaching that path needs an outstation whose retransmission timer is of the same order as the hold; on this relay it is about three orders larger. The mechanism forwards late copies, and the live-window case stays open.

A Lost OPERATE Cannot Be Repaired. The control lane has a second and worse case, which we found by reading the program rather than by measuring it. After the switch releases an OPERATE toward the relay it keeps that command’s application generation as a spent marker, and clears it only when the next SELECT arrives. A later OPERATE carrying the same generation is classified a duplicate and dropped. If the released command is lost on the relay-facing link, the master’s transport repairs the loss by retransmitting exactly those bytes, with the same generation, and the switch discards the packet the repair depends on. Forwarding a command out of the switch is not delivery to the relay, and the program cannot tell the two apart because the relay-facing link is not instrumented. No capture in this work exercises the path: the campaign’s 5,280 OPERATE exchanges lost nothing and every control operation returned success, which is the normal case working rather than evidence that the failure case is handled. We therefore claim no loss recovery on the control path, and exactly-once execution is neither provided nor claimed.

The Tail Is Not Eliminated. A fixed read-path release budget leaves about one exchange in a thousand answering too late to be scheduled, and a comparable fraction of protected READ exchanges more than 1 ms from the configured value. RO1 is a claim about the distribution, not about every transaction in it.

VII. RELATED WORK

Device fingerprinting. Devices can be identified from what they emit, without access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock in TCP timestamps [19], and GTID fingerprints a device and its type from the timing signature of its packets [20]. These works define the threat we address, each reading a different observable, a clock’s skew in one case and a device’s whole packet-timing signature in the other, and we remove neither. Instead, this work removes one interval inside a request-response exchange, the one the cross-layer response-time method of Formby et al. uses, and removes it from the master-facing observation only. Since their target remains the device type, whereas the classifier we evaluate separates three transaction classes on one outstation, we do not demonstrate that two devices become indistinguishable.

ICS and power-system fingerprinting. Formby et al. brought fingerprinting to control systems with the cross-layer response time and the physical operation time, measured at the device, of DNP3 outstations [10], and Gu et al. added device physics as a feature [21]. Jeon et al. fingerprint SCADA devices passively without deep packet inspection [22]. The reconnaissance these techniques serve enables false-data injection and process-control attacks [15], [23].

The anti-reconnaissance defenses closest to ours work on what the traffic says and on which nodes it reaches. DefRec virtualizes physical functions to present deceptive content [24], RAINCOAT randomizes data acquisition and injects decoy measurements [25], and a companion testbed evaluates such approaches on grid infrastructure [26]. Compared to these, our framework works below the semantics they reshape and complements them, changing when the wire carries the outstation’s answer rather than what the answer says.

General traffic obfuscation. Traffic morphing transforms one class’s packet-size distribution into another’s [3], while Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet size and rate at a bandwidth cost [6], and WTF-PAD pads inter-packet gaps adaptively [27]. These defenses remain effective against the size features they target, and the attacks that follow them have become correspondingly strong [5], [28].

The same size-and-timing channel identifies smart-home devices under encryption, where shaping is the countermeasure [29], and Pacer and NetShaper shape a tenant’s traffic to a schedule that bounds side-channel leakage in the cloud [30], [31]. Pacer and NetShaper each shape from a host or hypervisor that can add cover traffic, and the smart-home setting assumes an encrypted payload. Consequently our setting differs in both respects, since it carries a plaintext protocol whose leaking feature remains one interval inside a request-response exchange, and no endpoint can be modified to help. The framework therefore adds no byte and runs entirely in the network.

Programmable data-plane traffic transformation. P4 defines the programmable parser and match-action model we use [13]. PIFO defines programmable scheduling at line rate [32], and SP-PIFO approximates it with strict-priority queues on today’s switches [18], which is the primitive our reservoirs use. Ditto pads and injects chaff to a fixed pattern at line rate [7], Minos morphs and schedules traffic on a programmable switch at low cost [8], and Securitas combines fragmentation and insertion, and realizes them across several data planes [9]. NetHide obfuscates topology in the data plane against link-flooding reconnaissance [33]. Closest to our setting, Hu et al. use programmable switches to enhance industrial-protocol security [34]. Consequently our framework differs from these in the observable it addresses, namely the master-visible timing of a DNP3 transaction, and in holding packets rather than adding or reshaping them.

DNP3 timing and security. East et al. catalogue attacks on DNP3 and the absence of encryption that makes its semantics visible [14], and specification-based and state-based intrusion detection has been adapted to DNP3 [35], [36], yet none of these changes the timing an observer sees.

On the control path we report a different quantity from Formby et al., the master-visible OPERATE response-to-acknowledgment interval, which remains a protocol-response timing feature rather than the payload-timestamp estimate of physical operation time they use [10]. Section III states why the two are not interchangeable and what we therefore do not

claim.

VIII. CONCLUSION

How long an outstation takes to answer has become a signature an observer can profile, and the relay leaking it cannot be modified. Consequently this paper presents a timing-obfuscation framework that moves the problem into the network, placing a switch in front of the outstation that holds the packets carrying the device’s timing and releases each at a deadline computed when the transaction begins. The read path, which is READ and the SELECT phase of select-before-operate, times that deadline from the outstation’s acknowledgment; OPERATE times it from the request. Consequently, the interval an observer sees is the operator’s value rather than the device’s.

Evaluating this on one physical relay shows the released interval concentrating on the configured value, its interquartile range falling from 2.78 to 0.006 ms and its standard deviation from 2.61 to 0.63 ms, while a classifier trained on that interval before deployment drops from 0.745 balanced accuracy to three-class chance. The framework protects the exchanges its budget can schedule and forwards the rest untouched.

Three bounds limit the result. An adversary who retrainers recovers to 0.782 by adding the request-to-acknowledgment interval, which the mechanism does not target. The evidence comes from one relay and one campaign, and it separates transaction classes, not device models. Since the switch’s own release instants were never captured, the residual around the configured value measures a difference between two releases, which we cannot place inside the switch. An instrumented build measures an internal blocker-termination interval of about 1.7 μ s, characterizing the mechanism without decomposing that residual. Attributing it would take the framework evaluated on outstations from several vendors and the release instants captured on both switch links.

OPEN SCIENCE

The evidence and the analysis are reproducible from the raw captures, archived with the extraction and statistics code, the figure generators, the pinned environment and the tests. One script rebuilds every number and the campaign figures; the histogram and model figures have their own generators and the manuscript its own build. A hash manifest covers each published artefact, and a publication gate compares a fresh rebuild against what the paper contains, byte for byte, so a discrepancy fails rather than passing silently. That is stricter than numerical agreement: the numbers reproduce wherever the pinned environment installs, while the last bits of a plotted coordinate need not agree across interpreter builds. The figures the page budget keeps out of the manuscript are published too.

The switch program is the exact P4 source that ran, with its compilation record; the relay’s firmware and vendor documentation are not ours to distribute. On acceptance the captures, the analysis pipeline and the switch source are published under an open license.

ETHICS CONSIDERATIONS

This work required no human subjects and no personal data. The experiments ran on a laboratory testbed we own: one SEL-751A protective relay, one Intel Tofino-1 switch and one master host, on an isolated network with no connection to any operational system. No production grid, utility network or third-party device was contacted at any point.

The relay is physical equipment whose outputs can operate a breaker, so every select-before-operate exchange in the evaluation addresses isolated control points through a guarded driver. No control operation in this paper moved a breaker or energized a circuit: each OPERATE is a protocol exchange the relay accepted and acknowledged, not a physical actuation.

The mechanism we describe remains defensive, removing a timing feature that an adversary would use for reconnaissance, and it neither creates a new attack nor discloses a vulnerability in the relay, the switch or the DNP3 protocol. One effect works against the defender. Holding the acknowledgment makes the request-to-acknowledgment interval more informative than it was, from 0.479 to 0.809 balanced accuracy for an adversary that retrains. An operator weighing deployment needs that number.

REFERENCES

- [1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the Cyber Attack on the Ukrainian Power Grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016.
- [2] R. Langner, "Stuxnet: Dissecting a Cyberwarfare Weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.
- [3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.
- [4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail," in *2012 IEEE Symposium on Security and Privacy (S&P)*, 2012.
- [5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.
- [6] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A Systematic Approach to Developing and Evaluating Website Fingerprinting Defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.
- [7] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN Traffic Obfuscation at Line Rate," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.
- [8] Z. Wang, Q. Li, G. Xie, D. Zhao, K. Li, Z. Fan, L. Ma, and Y. Jiang, "Minos: A Lightweight and Dynamic Defense against Traffic Analysis in Programmable Data Planes," in *Proceedings of the 2025 USENIX Annual Technical Conference (ATC)*, 2025.
- [9] G. Xie, Q. Li, Z. Shi, G. Antichi, Y. Zhu, K. Li, C. Weng, S. Miano, Y. Jiang, and M. Xu, "Defending against Traffic Analysis Attacks with Flexible In-Network Obfuscation," in *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, pp. 2043–2063.
- [10] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, "Who's in Control of Your Control System? Device Fingerprinting for Cyber-Physical Systems," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016.
- [11] IEEE, "IEEE Standard for Electric Power Systems Communications—Distributed Network Protocol (DNP3)," 2012.
- [12] W. Eddy, "Transmission Control Protocol (TCP)," 2022.
- [13] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [14] S. East, J. Butts, M. Papa, and S. Shenoi, "A Taxonomy of Attacks on the DNP3 Protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol. 311)*. Springer, 2009, pp. 67–81.
- [15] Y. Liu, P. Ning, and M. K. Reiter, "False Data Injection Attacks against State Estimation in Electric Power Grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.
- [16] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's Retransmission Timer," 2011.
- [17] Schweitzer Engineering Laboratories, *SEL-751A Feeder Protection Relay Instruction Manual*, 2013, pM751A-01-NB, 20130329 printing; settings table, `STIME01 select/operate time-out`.
- [18] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating Push-In First-Out Behaviors using Strict-Priority Queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.
- [19] T. Kohno, A. Broido, and K. C. Claffy, "Remote Physical Device Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.
- [20] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A Technique for Physical Device and Device Type Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.
- [21] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for Cyber-Physical System Security: Device Physics Matters Too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, Sep. 2018.
- [22] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, "Passive Fingerprinting of SCADA in Critical Infrastructure Network without Deep Packet Inspection," arXiv:1608.07679 [cs.CR], Aug. 2016.
- [23] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks Against Process Control Systems: Risk Assessment, Detection, and Response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.
- [24] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing Physical Function Virtualization to Disrupt Reconnaissance of Power Grids' Cyber-Physical Infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.
- [25] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of Network Communication in Power Grid Cyber Infrastructure to Mislead Attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.
- [26] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-Physical Testbed: Case Study to Evaluate Anti-Reconnaissance Approaches on Power Grids' Cyber-Physical Infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.
- [27] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an Efficient Website Fingerprinting Defense," in *Computer Security – ESORICS 2016*, 2016.
- [28] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website Fingerprinting at Internet Scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.
- [29] N. Aphorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the Smart Home Private with Smart(er) IoT Traffic Shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019.
- [30] A. Mehta, M. Alzayat, R. D. Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive Network Side-Channel Mitigation in the Cloud," in *Proceedings of the 31st USENIX Security Symposium*, 2022.
- [31] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A Differentially Private Network Side-Channel Mitigation System," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [32] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, Agrawal, Anurag, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable Packet Scheduling at Line Rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.
- [33] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and Practical Network Topology Obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.
- [34] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial Network Protocol Security Enhancement Using Programmable Switches," in *Proceedings of the IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.
- [35] H. Lin, A. Slagell, C. D. Martino, Z. Kalbarczyk, and R. K. Iyer, "Adapting Bro into SCADA: Building a Specification-Based Intrusion Detection System for the DNP3 Protocol," in *Proceedings of the Eighth Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.
- [36] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 State-Based Intrusion Detection System," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.